

Documentación de Archivos del Proyecto HTTP Server

Archivos de Configuración del Proyecto

Cargo.toml

Propósito: Manifiesto del proyecto Rust que define metadatos, dependencias y configuraciones de compilación.

Contenido esperado:

- Información del paquete (nombre, versión, edición)
- Dependencias principales: clap, serde, serde_json, log, sha2, regex, flate2, chrono, rand
- Dependencias de desarrollo: criterion, tempfile, tarpaulin
- Features opcionales (async-runtime)
- Configuración de binarios y benchmarks
- Configuración de optimizaciones para release

Requerimientos:

- Versiones específicas de dependencias para garantizar compatibilidad
- Features mínimas necesarias para funcionalidad completa
- Separación clara entre dependencias de runtime y desarrollo

Cargo.lock

Propósito: Archivo generado automáticamente que fija las versiones exactas de todas las dependencias.

Contenido esperado:

- Versiones exactas de todas las dependencias directas e indirectas
- Checksums para verificación de integridad

Requerimientos:

- Debe estar versionado para garantizar builds reproducibles
- Actualización controlada mediante `cargo update`

README.md

Propósito: Documentación principal del proyecto para usuarios y desarrolladores.

Contenido esperado:

- Descripción del proyecto y objetivos
- Instrucciones de instalación y compilación
- Guía de uso básico con ejemplos
- Documentación de parámetros CLI
- Instrucciones para ejecutar tests y benchmarks
- Información sobre arquitectura general

- Guía de contribución

Requerimientos:

- Comandos ejecutables y verificados
- Ejemplos funcionales de uso
- Enlaces a documentación técnica adicional

.gitignore

Propósito: Especifica archivos y directorios que Git debe ignorar.

Contenido esperado:

- `/target/` (directorio de compilación de Rust)
- `Cargo.lock` (solo para librerías, incluido para binarios)
- Archivos temporales de prueba
- Logs y archivos de datos generados
- Archivos específicos del IDE

Código Fuente Principal (`src/`)

`main.rs`

Propósito: Punto de entrada del ejecutable del servidor.

Contenido esperado:

- Parsing de argumentos CLI usando clap
- Inicialización del sistema de logging
- Configuración del servidor basada en parámetros
- Manejo de señales del sistema (SIGINT, SIGTERM)
- Startup y shutdown graceful del servidor

Requerimientos:

- Validación robusta de parámetros de entrada
- Manejo de errores de inicialización
- Logging apropiado de eventos de startup/shutdown

`lib.rs`

Propósito: Raíz de la librería, exponiendo la API pública del proyecto.

Contenido esperado:

- Declaración de módulos públicos
- Re-exportación de tipos y funciones principales
- Documentación a nivel de crate
- Configuración de features del crate

Requerimientos:

- API limpia y bien documentada
- Exposición mínima necesaria (principio de menor privilegio)

Módulo de Configuración (`src/config/`)

`config/mod.rs`

Propósito: Declaración del módulo de configuración y exportación de tipos públicos.

Contenido esperado:

- Declaración de submódulos (`cli`, `settings`)
- Re-exportación de estructuras de configuración principales
- Documentación del módulo

`config/cli.rs`

Propósito: Definición y parsing de argumentos de línea de comandos.

Contenido esperado:

- Estructura `CliArgs` con derivado `clap::Parser`
- Definición de todos los parámetros CLI requeridos:
 - `--port`: Puerto del servidor
 - `--workers.{task}={count}`: Número de workers por tipo de tarea
 - `--queue.{task}={depth}`: Profundidad de colas por tarea
 - `--timeout.{type}={ms}`: Timeouts por tipo de operación
 - `--data-dir`: Directorio para archivos de datos
- Validación de parámetros
- Valores por defecto razonables

Requerimientos:

- Validación inmediata de rangos válidos
- Help messages claros y descriptivos
- Compatibilidad con variables de entorno equivalentes

`config/settings.rs`

Propósito: Estructura de configuración centralizada del servidor.

Contenido esperado:

- Estructura `ServerSettings` que consolida toda la configuración
- Método `from_cli()` para construir desde argumentos CLI
- Método `from_env()` para cargar desde variables de entorno
- Configuración por defecto para development/production
- Validación de configuración completa

Requerimientos:

- Inmutabilidad después de inicialización

- Validación cruzada de parámetros
- Serialización para debugging

Módulo del Servidor (`src/server/`)

`server/mod.rs`

Propósito: Declaración del módulo servidor y tipos públicos.

Contenido esperado:

- Declaración de submódulos
- Re-exportación de `HttpServer` y tipos relacionados
- Documentación del módulo

`server/http_server.rs`

Propósito: Implementación principal del servidor HTTP.

Contenido esperado:

- Estructura `HttpServer` con socket listener
- Método `new()` para inicialización
- Método `start()` para comenzar a aceptar conexiones
- Método `shutdown()` para cierre graceful
- Loop principal de aceptación de conexiones
- Delegación de conexiones al router
- Manejo de errores de red

Requerimientos:

- Soporte para múltiples conexiones concurrentes
- Manejo robusto de errores de socket
- Logging detallado de eventos de conexión
- Límites de conexiones concurrentes

`server/connection.rs`

Propósito: Manejo individual de conexiones HTTP.

Contenido esperado:

- Estructura `Connection` que encapsula un stream TCP
- Método `handle()` para procesar una conexión completa
- Lectura de datos del socket con timeouts
- Delegación al parser HTTP
- Envío de respuesta al cliente
- Cleanup de recursos

Requerimientos:

- Timeout de lectura/escritura configurable

- Manejo de conexiones malformadas
- Logging con request ID único
- Métricas de tiempo de conexión

server/request.rs

Propósito: Parsing de requests HTTP/1.0.

Contenido esperado:

- Estructura `HttpRequest` con método, path, headers, query params
- Parser completo de HTTP/1.0 (request line, headers)
- Extracción y parsing de query parameters
- Validación de formato HTTP
- Soporte para Content-Length en POST (opcional)

Requerimientos:

- Parsing robusto que rechace requests malformados
- Manejo de casos edge (headers largos, etc.)
- Extracción eficiente de parámetros
- Compatibilidad estricta con HTTP/1.0

server/response.rs

Propósito: Construcción de respuestas HTTP.

Contenido esperado:

- Estructura `HttpResponse` con status, headers, body
- Builder pattern para construcción de respuestas
- Serialización a formato HTTP/1.0 wire protocol
- Headers estándar (Content-Type, Content-Length, X-Request-Id)
- Respuestas de error predefinidas (400, 404, 500, etc.)

Requerimientos:

- Generación correcta de formato HTTP/1.0
- Headers de trazabilidad (X-Request-Id, X-Worker-Pid)
- Content-Length automático para el body
- Escape apropiado de caracteres especiales

server/router.rs

Propósito: Routing de requests a handlers apropiados.

Contenido esperado:

- Estructura `Router` con tabla de rutas
- Registro de handlers por path pattern
- Matching de rutas con parámetros

- Delegación a handler apropiado
- Manejo de rutas no encontradas (404)
- Logging de routing decisions

Requerimientos:

- Matching exacto de paths especificados en el proyecto
- Extracción de query parameters
- Routing thread-safe
- Métricas por endpoint

Módulo de Workers (`src/workers/`)

`workers/mod.rs`

Propósito: Declaración del módulo workers y exports principales.

`workers/worker_pool.rs`

Propósito: Pool de workers para un tipo específico de tarea.

Contenido esperado:

- Estructura `WorkerPool<T>` genérica por tipo de tarea
- Vector de worker threads
- Channel para distribución de trabajo
- Método `new()` para inicializar pool con N workers
- Método `submit()` para enviar trabajo
- Método `shutdown()` para terminación graceful
- Métricas de utilización del pool

Requerimientos:

- Thread safety completo
- Balanceo de carga entre workers
- Manejo de worker failures
- Recuperación automática de workers crashed

`workers/worker_manager.rs`

Propósito: Gestión centralizada de todos los pools de workers.

Contenido esperado:

- Estructura `WorkerManager` que contiene múltiples pools
- Mapa de TaskType -> WorkerPool
- Inicialización de pools basada en configuración
- Método `submit_task()` que dirige trabajo al pool correcto
- Recolección de métricas agregadas
- Shutdown coordinado de todos los pools

Requerimientos:

- Configuración flexible de workers por tipo
- Métricas agregadas por tipo de tarea
- Manejo de backpressure cuando pools están saturados

workers/task_queue.rs

Propósito: Cola thread-safe para tareas pendientes.

Contenido esperado:

- Estructura `TaskQueue<T>` con MPSC channels
- Método `push()` para encolar tareas
- Método `pop()` bloqueante para workers
- Método `try_pop()` no-bloqueante
- Método `len()` para métricas de tamaño
- Manejo de shutdown/poison

Requerimientos:

- Thread safety garantizado
- Comportamiento FIFO
- Capacidad limitada con backpressure
- Métricas de throughput

workers/worker_types.rs

Propósito: Definición de tipos y traits para el sistema de workers.

Contenido esperado:

- Enum `TaskType` (CPU, IO, Basic)
- Trait `Task` para tareas ejecutables
- Estructura `TaskResult` para resultados
- Enum `WorkerStatus` (Idle, Busy, Shutdown)
- Traits para workers especializados

Requerimientos:

- Serialización de tipos para métricas
- Implementación de Clone/Debug donde apropiado
- Type safety para diferentes tipos de tareas

Módulo de Jobs (`src/jobs/`)**jobs/mod.rs**

Propósito: Declaración del módulo jobs y tipos públicos.

jobs/job_manager.rs

Propósito: Gestor principal del sistema de jobs asíncronos.

Contenido esperado:

- Estructura `JobManager` como singleton/servicio global
- Método `submit_job()` para crear nuevos jobs
- Método `get_job_status()` para consultar estado
- Método `get_job_result()` para obtener resultados
- Método `cancel_job()` para cancelación
- Cleanup automático de jobs completados/expirados

Requerimientos:

- Thread safety completo
- Persistencia efímera de metadatos
- Timeouts configurables por tipo de job
- Límites de jobs concurrentes

`jobs/job_queue.rs`

Propósito: Cola de jobs con soporte para prioridades.

Contenido esperado:

- Estructura `JobQueue` con heap binario por prioridad
- Método `enqueue()` con prioridad
- Método `dequeue()` respetando prioridades
- Método `peek()` para inspección sin consumir
- Soporte para FIFO dentro de misma prioridad

Requerimientos:

- Implementación eficiente ($O(\log n)$ enqueue/dequeue)
- Thread safety
- Métricas por prioridad
- Backpressure cuando cola está llena

`jobs/job_storage.rs`

Propósito: Persistencia efímera de metadatos de jobs.

Contenido esperado:

- Trait `JobStorage` para abstracción
- Implementación `FileStorage` con archivos temporales
- Implementación `MemoryStorage` para testing
- Serialización/deserialización de job state
- Cleanup automático en shutdown

Requerimientos:

- Supervivencia a restart graceful

- Formato de datos versionado
- Recovery de jobs en progreso tras restart
- Garbage collection de jobs antiguos

jobs/job_scheduler.rs

Propósito: Planificador que asigna jobs a workers disponibles.

Contenido esperado:

- Estructura `JobScheduler` con políticas de scheduling
- Implementación FIFO por defecto
- Soporte para prioridades (low/normal/high)
- Balanceador de carga entre workers
- Respect de límites de concurrencia por tipo

Requerimientos:

- Políticas de scheduling configurables
- Fairness entre diferentes tipos de jobs
- Respeto de límites de recursos
- Métricas de latencia de scheduling

jobs/job_types.rs

Propósito: Definición de tipos para el sistema de jobs.

Contenido esperado:

- Estructura `Job` con ID, tipo, parámetros, estado
- Enum `JobStatus` (Queued, Running, Done, Error, Canceled)
- Enum `JobPriority` (Low, Normal, High)
- Estructura `JobResult` con datos de resultado
- UUID generation para job IDs

Requerimientos:

- Serialización completa para persistencia
- Job IDs únicos y no predecibles
- Timestamps para todas las transiciones de estado
- Metadatos para ETA calculations

Módulo de Handlers (`src/handlers/`)

handlers/mod.rs

Propósito: Declaración del módulo handlers y re-exports.

handlers/basic.rs

Propósito: Implementación de endpoints básicos del servidor.

Contenido esperado:

- Funciones handler para cada endpoint básico:
 - `fibonacci()`: Cálculo de secuencia Fibonacci
 - `createfile()`: Creación de archivos con contenido repetido
 - `deletefile()`: Eliminación de archivos
 - `reverse()`: Inversión de strings
 - `toupper()`: Conversión a mayúsculas
 - `random()`: Generación de números aleatorios
 - `timestamp()`: Timestamp actual
 - `hash()`: Hash de strings
 - `simulate()`: Simulación de trabajo
 - `sleep()`: Delay configurable
 - `loadtest()`: Test de carga interno
 - `help()`: Documentación de endpoints

Requerimientos:

- Validación estricta de parámetros de entrada
- Manejo robusto de casos edge
- Respuestas JSON consistentes
- Logging de operaciones
- Timeouts apropiados

handlers/cpu_intensive.rs

Propósito: Handlers para tareas CPU-intensive.

Contenido esperado:

- `isprime()`: Test de primalidad con Miller-Rabin
- `factor()`: Factorización en números primos
- `pi()`: Cálculo de π con algoritmos iterativos
- `mandelbrot()`: Generación de conjunto de Mandelbrot
- `matrixmul()`: Multiplicación de matrices grandes

Requerimientos:

- Algoritmos eficientes y correctos
- Soporte para cancelación de tareas largas
- Progress reporting para jobs asíncronos
- Validación de rangos de entrada
- Métricas de tiempo de ejecución

handlers/io_intensive.rs

Propósito: Handlers para tareas I/O-intensive.

Contenido esperado:

- `sortfile()`: Ordenamiento de archivos grandes

- `wordcount()`: Conteo de líneas/palabras/bytes
- `grep()`: Búsqueda de patrones con regex
- `compress()`: Compresión de archivos (gzip/xz)
- `hashfile()`: Cálculo de hash de archivos

Requerimientos:

- Soporte para archivos $\geq 50\text{MB}$
- Streaming processing para archivos grandes
- Manejo robusto de errores de I/O
- Progress reporting para operaciones largas
- Cleanup de archivos temporales

handlers/job_endpoints.rs

Propósito: Endpoints para gestión del sistema de jobs.

Contenido esperado:

- `jobs_submit()`: Creación de nuevos jobs
- `jobs_status()`: Consulta de estado de jobs
- `jobs_result()`: Obtención de resultados
- `jobs_cancel()`: Cancelación de jobs
- `jobs_list()`: Listado de jobs (para debugging)

Requerimientos:

- Validación de job IDs
- Manejo de jobs no existentes
- Rate limiting de submissions
- Respuestas consistentes con especificación

handlers/metrics.rs

Propósito: Endpoints de métricas y status del servidor.

Contenido esperado:

- `status()`: Estado general del servidor (uptime, PID, conexiones)
- `metrics()`: Métricas detalladas (latencias, throughput, colas)
- Recolección de estadísticas de todos los subsistemas
- Cálculo de percentiles (p50, p95, p99)

Requerimientos:

- Formato JSON consistente según especificación
- Métricas en tiempo real
- Historial de métricas (ventana deslizante)
- Performance mínimo impact en recolección

handlers/handler_traits.rs

Propósito: Traits comunes y utilidades para handlers.

Contenido esperado:

- Trait `RequestHandler` para handlers genéricos
- Trait `AsyncHandler` para operaciones largas
- Funciones de utilidad para parsing de parámetros
- Helpers para construcción de respuestas JSON
- Macros para reducir boilerplate

Módulo de Algoritmos (`src/algorithms/`)

`algorithms/mod.rs`

Propósito: Declaración del módulo `algorithms`.

`algorithms/prime.rs`

Propósito: Algoritmos relacionados con números primos.

Contenido esperado:

- Implementación Miller-Rabin para test de primalidad
- Algoritmo de factorización (trial division optimizada)
- Función `is_prime()` configurable entre métodos
- Función `factorize()` que retorna factores y exponentes

Requerimientos:

- Algoritmos matemáticamente correctos
- Optimización para números grandes
- Configurabilidad de precisión/velocidad
- Soporte para cancelación temprana

`algorithms/pi_calculation.rs`

Propósito: Algoritmos para cálculo de π .

Contenido esperado:

- Implementación Spigot algorithm
- Implementación Chudnovsky (opcional)
- Control de precisión (número de dígitos)
- Control de tiempo máximo de ejecución

Requerimientos:

- Precisión arbitraria usando librerías apropiadas
- Iterative approach para permitir cancelación
- Progress reporting por número de dígitos calculados
- Validación de resultados conocidos

algorithms/mandelbrot.rs

Propósito: Generación del conjunto de Mandelbrot.

Contenido esperado:

- Función `generate_mandelbrot()` que produce matriz de iteraciones
- Parámetros configurables: width, height, max_iterations
- Región compleja configurable (zoom)
- Optimizaciones de performance

Requerimientos:

- Salida en formato JSON como matriz 2D
- Soporte para diferentes resoluciones
- Algoritmo matemáticamente correcto
- Posible paralelización interna

algorithms/matrix_ops.rs

Propósito: Operaciones con matrices.

Contenido esperado:

- Generación de matrices pseudoaleatorias con seed
- Multiplicación de matrices optimizada
- Función `matrix_multiply()` para matrices NxN
- Hash SHA-256 del resultado para verificación

Requerimientos:

- Implementación eficiente ($O(n^3)$ clásica o Strassen)
- Reproducibilidad con mismo seed
- Manejo de matrices grandes (memory management)
- Correctness verification con casos conocidos

algorithms/sorting.rs

Propósito: Algoritmos de ordenamiento para archivos.

Contenido esperado:

- Implementación merge sort para archivos grandes
- Implementación quicksort in-memory
- External sorting para archivos que no caben en memoria
- Función `sort_file()` que delega al algoritmo apropiado

Requerimientos:

- Soporte para archivos ≥ 50 MB
- Merge sort estable
- Memory-efficient external sorting

- Progress reporting para archivos grandes

Módulo de Operaciones I/O (`src/io_operations/`)

`io_operations/mod.rs`

Propósito: Declaración del módulo `io_operations`.

`io_operations/file_ops.rs`

Propósito: Operaciones básicas con archivos.

Contenido esperado:

- Funciones para creación/eliminación de archivos
- Lectura/escritura streaming para archivos grandes
- Validación de paths y permisos
- Creación de archivos temporales

Requerimientos:

- Manejo seguro de paths (evitar directory traversal)
- Error handling robusto para operaciones de I/O
- Cleanup automático de recursos
- Soporte para archivos grandes (streaming)

`io_operations/compression.rs`

Propósito: Compresión y descompresión de archivos.

Contenido esperado:

- Función `compress_gzip()` para compresión gzip
- Función `compress_xz()` para compresión xz
- Streaming compression para archivos grandes
- Métricas de ratio de compresión

Requerimientos:

- Implementación streaming (no cargar archivo completo en memoria)
- Manejo de archivos de cualquier tamaño
- Progress reporting para archivos grandes
- Cleanup de archivos temporales en caso de error

`io_operations/file_processing.rs`

Propósito: Procesamiento de archivos (word count, grep).

Contenido esperado:

- Función `word_count()` compatible con comando `wc`
- Función `grep_file()` con soporte para regex

- Procesamiento streaming línea por línea
- Optimizaciones para archivos de texto grandes

Requerimientos:

- Compatibilidad con formatos de texto diversos (UTF-8)
- Regex engine eficiente
- Memory-bounded processing
- Resultados idénticos a herramientas estándar Unix

io_operations/hashing.rs

Propósito: Cálculo de hashes de archivos.

Contenido esperado:

- Función `hash_file_sha256()` para SHA-256
- Soporte para otros algoritmos (SHA-1, MD5) si requerido
- Streaming hashing para archivos grandes
- Formato hexadecimal de salida

Requerimientos:

- Implementación streaming (no cargar archivo en memoria)
- Resultados idénticos a herramientas como `shasum`
- Performance optimizado
- Validación con test vectors conocidos

Módulo de Utilidades (`src/utils/`)

`utils/mod.rs`

Propósito: Declaración del módulo `utils`.

`utils/json.rs`

Propósito: Utilidades para construcción de JSON.

Contenido esperado:

- Helpers para construcción manual de JSON
- Wrappers sobre `serde_json` para casos comunes
- Funciones para respuestas de error estándar
- Validación de JSON generado

Requerimientos:

- JSON válido en todos los casos
- Escape apropiado de caracteres especiales
- Performance aceptable para respuestas frecuentes
- Consistencia en formato de respuestas

utils/logging.rs

Propósito: Sistema de logging centralizado.

Contenido esperado:

- Configuración de env_logger
- Macros personalizadas para logging estructurado
- Request ID tracking a través de requests
- Log levels configurables

Requerimientos:

- Thread safety completo
- Rotation de logs automática
- Structured logging para parseo automático
- Performance overhead mínimo

utils/metrics.rs

Propósito: Recolección y agregación de métricas.

Contenido esperado:

- Estructura `MetricsCollector` thread-safe
- Tracking de latencias con histogramas
- Cálculo de percentiles (p50, p95, p99)
- Métricas de throughput y error rates
- Reset/snapshot functionality

Requerimientos:

- Thread safety para updates concurrentes
- Memory-bounded (rolling windows)
- Performance overhead mínimo
- Precisión estadística apropiada

utils/crypto.rs

Propósito: Utilidades criptográficas.

Contenido esperado:

- Wrappers sobre SHA-256 de la librería sha2
- Generación de request IDs únicos
- Funciones de hash para strings
- HMAC si requerido para autenticación

Requerimientos:

- Uso de librerías criptográficas establecidas
- No implementar crypto primitives desde cero

- Thread safety para operaciones concurrentes
- Test vectors para verificar correctness

utils/validation.rs

Propósito: Validación de parámetros de entrada.

Contenido esperado:

- Funciones de validación para cada tipo de parámetro
- Validadores composables para parámetros complejos
- Generación de mensajes de error útiles
- Whitelist/blacklist de valores permitidos

Requerimientos:

- Validación exhaustiva según especificaciones
- Mensajes de error claros y específicos
- Performance para validaciones frecuentes
- Prevention de ataques de injection

Módulo de Errores (src/error/)

error/mod.rs

Propósito: Declaración del módulo error.

error/types.rs

Propósito: Definición de tipos de error del sistema.

Contenido esperado:

- Enum `ServerError` con variantes para cada tipo de error
- Implementación de `std::error::Error` trait
- Conversión automática desde errores de librerías externas
- Funciones helper para crear errores con contexto

Requerimientos:

- Hierarchy clara de tipos de error
- Información suficiente para debugging
- Serialización a JSON para respuestas HTTP
- Integration con el sistema de logging

Tests (tests/)

tests/common/mod.rs

Propósito: Código común compartido entre tests.

tests/common/test_server.rs

Propósito: Helper para levantar servidor de prueba.

Contenido esperado:

- Función `start_test_server()` en puerto aleatorio
- Configuración minimal para tests
- Cleanup automático al finalizar tests
- Helpers para setup de datos de prueba

Requerimientos:

- Aislamiento completo entre tests
- Puertos aleatorios para evitar conflictos
- Timeout apropiado para startup/shutdown
- Cleanup garantizado incluso si test falla

`tests/common/client.rs`

Propósito: Cliente HTTP simple para tests.

Contenido esperado:

- Cliente HTTP básico usando `std::net`
- Funciones helper para GET requests comunes
- Parsing básico de respuestas HTTP
- Timeout configurable para requests

Requerimientos:

- Implementación simple sin dependencias externas
- Compatibilidad con HTTP/1.0
- Manejo de errors de red
- Reusabilidad entre diferentes tests

`tests/integration_tests.rs`

Propósito: Tests de integración principales.

Contenido esperado:

- Test para cada endpoint definido en la especificación
- Tests de parámetros válidos e inválidos
- Tests de casos edge y límites
- Tests de respuestas HTTP correctas (códigos, headers)

Requerimientos:

- Cobertura completa de todos los endpoints
- Verificación de formato JSON de respuestas
- Tests de manejo de errores
- Assertions claras y específicas

tests/load_tests.rs

Propósito: Tests de carga y concurrencia.

Contenido esperado:

- Tests con N clientes concurrentes
- Verificación de ausencia de race conditions
- Tests de saturación de worker pools
- Medición de latencias bajo carga

Requerimientos:

- Escalabilidad configurable (número de clientes)
- Detección de deadlocks
- Métricas de performance reproducibles
- Cleanup apropiado después de load tests

tests/job_tests.rs

Propósito: Tests específicos del sistema de jobs.

Contenido esperado:

- Tests del ciclo completo de vida de jobs
- Tests de cancelación de jobs
- Tests de persistencia efímera
- Tests de prioridades y scheduling

Requerimientos:

- Tests de edge cases (jobs no existentes, etc.)
- Verificación de timeouts
- Tests de recovery después de restart
- Concurrencia en gestión de jobs

tests/performance_tests.rs

Propósito: Tests de performance y benchmarks.

Contenido esperado:

- Benchmarks para algoritmos CPU-intensive
- Benchmarks para operaciones I/O
- Comparación de performance entre algoritmos
- Regression testing de performance

Requerimientos:

- Resultados reproducibles
- Statistical significance
- Baseline comparisons

- CI integration para detectar regressions

Benchmarks ([benches/](#))

[benches/io_benchmarks.rs](#)

Propósito: Benchmarks para operaciones I/O-intensive.

Contenido esperado:

- Benchmarks para ordenamiento de archivos
- Benchmarks para compresión/descompresión
- Benchmarks para operaciones de hashing
- Benchmarks para grep y word count
- Variación de tamaños de archivo (1MB, 10MB, 50MB+)

Requerimientos:

- Datos de prueba consistentes y reproducibles
- Medición separada de I/O vs CPU time
- Benchmarks con diferentes tamaños de archivo
- Cleanup automático de archivos temporales

[benches/concurrency_benchmarks.rs](#)

Propósito: Benchmarks de concurrencia y throughput.

Contenido esperado:

- Benchmarks de throughput con diferentes números de workers
- Benchmarks de latencia vs load
- Benchmarks de escalabilidad horizontal
- Contention benchmarks para recursos compartidos

Requerimientos:

- Medición de throughput (requests/second)
- Medición de latencia (p50, p95, p99)
- Variación sistemática de parámetros de concurrencia
- Statistical significance en resultados

Datos de Prueba ([data/](#))

[data/generators/generate_test_files.rs](#)

Propósito: Generación determinística de archivos de prueba.

Contenido esperado:

- Función [generate_numbers_file\(\)](#) para archivos de números enteros
- Función [generate_text_file\(\)](#) para archivos de texto
- Función [generate_binary_file\(\)](#) para datos binarios

- Seeds configurables para reproducibilidad
- Generación eficiente de archivos grandes (50MB+)

Requerimientos:

- Reproducibilidad completa con mismo seed
- Generación eficiente sin consumir memoria excesiva
- Formatos válidos para cada tipo de prueba
- Validación de archivos generados

data/samples/numbers_50mb.txt

Propósito: Archivo de muestra con números enteros para pruebas de ordenamiento.

Contenido esperado:

- Archivo de exactamente ~50MB
- Números enteros, uno por línea
- Distribución pseudoaleatoria conocida
- Casos especiales incluidos (números grandes, negativos)

Requerimientos:

- Formato compatible con algoritmos de ordenamiento
- Tamaño suficiente para tests de memoria
- Contenido validado y verificable

data/samples/sample_data.json

Propósito: Datos JSON de muestra para tests varios.

Contenido esperado:

- Estructura JSON compleja para parsing tests
- Casos edge (strings con caracteres especiales)
- Arrays y objetos anidados
- Unicode content

Requerimientos:

- JSON válido y bien formado
- Cobertura de casos especiales de JSON
- Tamaño manejable para tests unitarios

Scripts Auxiliares (scripts/)**scripts/build.sh**

Propósito: Script de compilación automatizada.

Contenido esperado:

- Compilación en modo debug y release
- Verificación de dependencias
- Limpieza de artifacts previos
- Validación de código con clippy
- Formateo automático con rustfmt

Requerimientos:

- Detección de errores de compilación
- Exit codes apropiados para CI
- Output claro de progreso y errores
- Cross-platform compatibility (Linux/macOS)

scripts/test.sh

Propósito: Ejecución automatizada de todos los tests.

Contenido esperado:

- Ejecución de tests unitarios
- Ejecución de tests de integración
- Generación de datos de prueba necesarios
- Reporte de cobertura de código
- Cleanup después de tests

Requerimientos:

- Ejecución en orden apropiado
- Captura de output de tests fallidos
- Reporte de cobertura $\geq 90\%$
- Timeout apropiado para tests largos

scripts/coverage.sh

Propósito: Generación de reportes de cobertura de código.

Contenido esperado:

- Uso de tarpaulin para coverage
- Generación de reporte HTML
- Verificación de umbral mínimo (90%)
- Exclusión de archivos de prueba del cálculo

Requerimientos:

- Reporte detallado línea por línea
- Formato HTML navegable
- Integration con CI para reporting automático
- Fallar si cobertura $< 90\%$

scripts/benchmark.sh

Propósito: Ejecución automatizada de benchmarks.

Contenido esperado:

- Ejecución de todos los benchmarks con criterion
- Generación de reportes de performance
- Comparación con baselines previos
- Detección de regresiones de performance

Requerimientos:

- Configuración de ambiente para benchmarks consistentes
- Reportes en formato machine-readable
- Statistical rigor en mediciones
- Archive de resultados históricos

Documentación ([docs/](#))

[docs/architecture.md](#)

Propósito: Descripción detallada de la arquitectura del sistema.

Contenido esperado:

- Diagrama de arquitectura general
- Descripción de cada componente principal
- Flujo de datos entre componentes
- Decisiones de diseño y trade-offs
- Patrones arquitectónicos utilizados

Requerimientos:

- Diagramas claros y actualizados
- Explicación de decisiones de concurrencia
- Documentación de interfaces entre módulos
- Rationale para elección de tecnologías

[docs/api_reference.md](#)

Propósito: Documentación completa de la API HTTP.

Contenido esperado:

- Especificación de cada endpoint
- Parámetros requeridos y opcionales
- Formatos de request y response
- Códigos de error y su significado
- Ejemplos de uso con curl

Requerimientos:

- Formato consistente para todos los endpoints

- Ejemplos ejecutables y verificados
- Documentación de headers especiales
- Especificación de límites y restricciones

docs/performance_analysis.md

Propósito: Análisis de performance del sistema.

Contenido esperado:

- Resultados de benchmarks
- Análisis de escalabilidad
- Identificación de bottlenecks
- Recomendaciones de tuning
- Comparación con sistemas similares

Requerimientos:

- Datos empíricos de performance
- Metodología clara de medición
- Análisis estadístico riguroso
- Recomendaciones prácticas de optimización

Ejemplos (examples/)

examples/client_examples.rs

Propósito: Ejemplos de código para clientes del servidor.

Contenido esperado:

- Cliente básico HTTP/1.0 en Rust
- Ejemplos de uso de cada endpoint principal
- Manejo de respuestas JSON
- Manejo de errores de red

Requerimientos:

- Código compilable y ejecutable
- Documentación inline clara
- Cobertura de casos comunes de uso
- Error handling ejemplar

examples/load_test_client.rs

Propósito: Cliente para tests de carga del servidor.

Contenido esperado:

- Cliente multi-threaded para generar carga
- Configuración de número de threads concurrentes
- Medición de latencias y throughput

- Generación de reportes de carga

Requerimientos:

- Escalabilidad configurable
- Medición precisa de métricas
- Distribución realista de requests
- Reporte estadístico de resultados

Archivos de Configuración de Herramientas

.gitignore

Contenido esperado:

```
/target/  
**/*.rs.bk  
*.pdb  
.env  
/data/generated/  
/logs/  
.DS_Store  
.vscode/  
.idea/  
*.log  
*.tmp  
/coverage/
```

Requerimientos:

- Exclusión de artifacts de build
- Exclusión de archivos temporales de prueba
- Exclusión de archivos específicos de IDE
- Inclusión de Cargo.lock para binarios

Consideraciones Generales de Implementación

Thread Safety

- Todos los módulos que manejan estado compartido deben usar primitivas de sincronización apropiadas (Arc, Mutex, RwLock, channels)
- Evitar uso de static mut y unsafe code excepto donde sea absolutamente necesario
- Documentar explícitamente assumptions de thread safety

Error Handling

- Uso consistente de Result<T, E> en toda la codebase
- Propagación de errores usando ? operator
- Logging apropiado de errores antes de propagación

- Mensajes de error útiles para usuarios finales

Performance

- Algoritmos con complejidad apropiada para casos de uso esperados
- Evitar allocaciones innecesarias en hot paths
- Uso de streaming para archivos grandes
- Profiling regular para identificar bottlenecks

Testing Strategy

- Tests unitarios para cada función pública
- Tests de integración para flujos completos
- Property-based testing para algoritmos complejos
- Mock objects para dependencias externas
- Tests de performance para prevenir regresiones

Documentation

- Documentación completa de API pública usando doc comments
- Ejemplos de código en documentación
- Architecture decision records para decisiones importantes
- Runbooks para operación y troubleshooting

Esta estructura modular permite desarrollo independiente de componentes, facilita testing exhaustivo, y proporciona separación clara de responsabilidades necesaria para un proyecto de esta complejidad y escala./cpu_benchmarks.rs` **Propósito:** Benchmarks para tareas CPU-intensive usando criterion.

Contenido esperado:

- Benchmarks para algoritmos de primos
- Benchmarks para cálculo de π
- Benchmarks para Mandelbrot
- Benchmarks para multiplicación de matrices

Requerimientos:

- Uso de criterion para statistical rigor
- Parámetros representativos de uso real
- Warmup apropiado
- Detección de outliers

`benches